

目录

- 引入 ffmpeg.....1
- 解码显示到窗口的 demo.....2
 - 0.类的创建.....2
 - 1.初始化.....2
 - 2.分配 AVFormatContext.....2
 - 3.打开视频文件并获取信息.....3
 - 4.查找文件中的视频流.....3
 - 5.查找解码器.....3
 - 6.申请解码需要的结构体.....4
 - 7. YUV 数据转换成 RGB32 所需要的存储.....4
 - 8.循环读取视频.....4
 - 9.回收数据.....5
 - 10.声明信号.....6
 - 简易同步.....6

引入 ffmpeg

Qt Creator 中修改工程配置文件 testFFmpeg.pro:
在里面加入:

```
INCLUDEPATH += $$PWD/ffmpeg-4.2.2/include

LIBS += $$PWD/ffmpeg-4.2.2/lib/avcodec.lib\
        $$PWD/ffmpeg-4.2.2/lib/avdevice.lib\
        $$PWD/ffmpeg-4.2.2/lib/avfilter.lib\
        $$PWD/ffmpeg-4.2.2/lib/avformat.lib\
        $$PWD/ffmpeg-4.2.2/lib/avutil.lib\
        $$PWD/ffmpeg-4.2.2/lib/postproc.lib\
        $$PWD/ffmpeg-4.2.2/lib/swresample.lib\
        $$PWD/ffmpeg-4.2.2/lib/swscale.lib
```

./ --- \$\$PWD

main.cpp 写成如下:

```
#include <iostream>

using namespace std;
extern "C"
{
#include "libavcodec/avcodec.h"
#include "libavformat/avformat.h"
```

```

#include "libswscale/swscale.h"
#include "libavdevice/avdevice.h"
}
///由于我们建立的是 C++的工程
///编译的时候使用的 C++的编译器编译
///而 FFMPEG 是 C 的库
///因此这里需要加上 extern "C"
///否则会提示各种未定义

int main(int argc, char *argv[])
{
    //这里简单的输出一个版本号
    cout << "Hello FFmpeg!" << endl;
    av_register_all();
    unsigned version = avcodec_version();
    cout << "version is:" << version;
    return 0;
}

```

解码显示到窗口的 demo

结合上面的过程,写一个解码显示到窗口的 demo
具体的步骤如下所示:

0.类的创建

由于解码视频是后台应该做的事情,那么应该单独引入一个线程来完成. 那么可以使用 qt 自带的线程类来完成. 创建 QThread 线程类.

1.初始化

首先需要先初始化一下, 使用如下函数:

//1. 初始化 FFMPEG 调用了这个才能正常适用编码器和解码器 注册所用函数

```
av_register_all();
```

使用这个函数完成编码器和解码器的初始化, 只有初始化了编码器和解码器才能正常使用, 否则会在打开编解码器的时候失败。

2.分配 AVFormatContext

接着需要分配一个 AVFormatContext, FFMPEG 所有操作都要通过 AVFormatContext 来进行.

//2. 需要分配一个 AVFormatContext, FFMPEG 所有的操作都要通过这个 AVFormatContext 来进行 可以理解为视频文件指针

```
AVFormatContext *pFormatCtx = avformat_alloc_context();
```

3.打开视频文件并获取信息

接着调用打开视频文件

//中文兼容

```
std::string path = m_fileName.toStdString();
```

```
const char* file_path = path.c_str();
```

打开视频文件

//3. 打开视频文件

```
if( avformat_open_input(&pFormatCtx, file_path, NULL, NULL) != 0 )
```

```
{
```

```
    qDebug()<<"can't open file";
```

```
    return;
```

```
}
```

//3.1 获取视频文件信息

```
if (avformat_find_stream_info(pFormatCtx, NULL) < 0)
```

```
{
```

```
    qDebug()<<"Could't find stream infomation.";
```

```
    return;
```

```
}
```

4.查找文件中的视频流

///循环查找视频中包含的流信息，直到找到视频类型的流

///便将其记录下来 保存到 videoStream 变量中

///这里我们现在只处理视频流 音频流先不管他

//4. 读取视频流

```
int videoStream = -1;
```

```
int i;
```

```
for ( i = 0; i < pFormatCtx->nb_streams; i++) {
```

```
    if (pFormatCtx->streams[i]->codec->codec_type == AVMEDIA_TYPE_VIDEO)
```

```
    {
```

```
        videoStream = i;
```

```
    }
```

```
}
```

///如果 videoStream 为-1 说明没有找到视频流

```
if (videoStream == -1) {
```

```
    qDebug()<< "Didn't find a video stream." ;
```

```
    return;
```

```
}
```

5.查找解码器

//5. 查找解码器

```
auto pCodecCtx = pFormatCtx->streams[videoStream]->codec;
```

```
auto pCodec = avcodec_find_decoder(pCodecCtx->codec_id);
```

```

if (pCodec == NULL) {
    qDebug()<< "Codec not found." ;
    return;
}
//打开解码器
if(avcodec_open2(pCodecCtx, pCodec, NULL) < 0) {
    qDebug()<< "Could not open codec." ;
    return;
}

```

可以看出我们可以直接根据查找到的视频流信息获取到解码器。

而且我们并不知道他实际用的是什么编码器。

这就是为什么一开始我们使用 FFMPEG 来操作，因为很多东西我们可以不关心。

6.申请解码需要的结构体

```

//6. 申请解码需要的结构体 AVFrame 视频缓存的结构体
AVFrame *pFrame, *pFrameRGB;
pFrame = av_frame_alloc();
pFrameRGB = av_frame_alloc();
int y_size = pCodecCtx->width * pCodecCtx->height;
AVPacket *packet = (AVPacket *) malloc(sizeof(AVPacket)); //分配一个 packet
av_new_packet(packet, y_size); //分配 packet 的数据

```

7. YUV 数据转换成 RGB32 所需要的存储

//7. 这里我们将解码后的 YUV 数据转换成 RGB32 YUV420p 格式视频数据-->RGB32--> 图片显示出来

```

struct SwsContext *img_convert_ctx;
img_convert_ctx = sws_getContext(pCodecCtx->width, pCodecCtx->height,
    pCodecCtx->pix_fmt, pCodecCtx->width, pCodecCtx->height,
    AV_PIX_FMT_RGB32, SWS_BICUBIC, NULL, NULL, NULL);
auto numBytes =
avpicture_get_size(AV_PIX_FMT_RGB32, pCodecCtx->width, pCodecCtx->height);
uint8_t * out_buffer = (uint8_t *) av_malloc(numBytes * sizeof(uint8_t));
avpicture_fill((AVPicture *) pFrameRGB, out_buffer, AV_PIX_FMT_RGB32,
    pCodecCtx->width, pCodecCtx->height);

```

8.循环读取视频

//8.循环读取视频帧，转换为 RGB 格式，抛出信号去控件显示

```

int ret, got_picture;
while(1)
{
    //可以看出 av_read_frame 读取的是一帧视频，并存入一个 AVPacket 的结构中
    if (av_read_frame(pFormatCtx, packet) < 0)
    {

```

```

        break; //这里认为视频读取完了
    }
    //生成图片
    if (packet->stream_index == videoStream)
    {
        // 解码 packet 存在 pFrame 里面
        ret = avcodec_decode_video2(pCodecCtx, pFrame, &got_picture, packet);

        if (ret < 0) {
            qDebug() << "decode error";
            return ;
        }

        //有解码器解码之后得到的图像数据都是 YUV420 的格式，而这里需要将其保存成图片文件
        //因此需要将得到的 YUV420 数据转换成 RGB 格式
        if (got_picture) {
            sws_scale(img_convert_ctx,
                      (uint8_t const * const *) pFrame->data,
                      pFrame->linesize, 0, pCodecCtx->height, pFrameRGB->data,
                      pFrameRGB->linesize);

            // 将 out_buffer 里面的数据存在 QImage 里面
            QImage tmpImg((uchar
*)out_buffer, pCodecCtx->width, pCodecCtx->height, QImage::Format_RGB32);
            //把图像复制一份 传递给界面显示
            //显示到控件 多线程 无法控制控件 所以要发射信号
            emit SIG_GetOneImage( tmpImg );
        }
    }
    av_free_packet(packet);
    msleep(5); // 停一停
}

```

至于 YUV420 和 RGB 图像格式的具体内容，这里不用去了解。这里只需要知道有这么个东西就行了，对我们使用 FFMPEG 转换没有影响。

9.回收数据

//9.回收数据

```

av_free(out_buffer);
av_free(pFrame);
av_free(pFrameRGB);
avcodec_close(pCodecCtx);
avformat_close_input(&pFormatCtx);

```

由于我们不能够在子线程中操作界面，(操作界面只能在主线程中进行，几乎所有的图形界面开发都是这样设定)，因此我们只能给主线程发送信号，信号带上这个 QImage，让主线程帮忙把这个图像显示出来。

10.声明信号

signals:

```
void SIG_GetOneFrame(QImage); //每次获取一帧画面，发送此信号
```

这样，我们可以在外面使用槽函数连接，得到这个图片，然后粘贴到控件上面。

简易同步

首先添加头文件

```
#include"libavutil/time.h"
```

在循环读取前加入语句:

```
int64_t start_time = av_gettime();
```

```
int64_t pts = 0; //当前视频帧的pts
```

加入方法如图:

```
int64_t start_time = av_gettime();  
int64_t pts = 0; //当前视频的pts
```

```
//8.循环读取视频帧，转换为RGB格式，抛出信号去控件显示
```

```
int ret, got_picture;
```

```
while(1)
```

```
{
```

```
    //可以看出 av_read_frame读取的是一帧视频，并存入一个AVPacket的结构中
```

```
    if (av_read_frame(pFormatCtx, packet) < 0)
```

```
    {
```

```
        break; //这里认为视频读取完了
```

```
    }
```

在解码之前加入语句，情况如下

```
int64_t realTime = av_gettime() - start_time; //主时钟时间
```

```
while(pts > realTime)
```

```
{
```

```
    msleep(10);
```

```
    realTime = av_gettime() - start_time; //主时钟时间
```

```
}
```

加入方法如图

```
while(1)
```

```
{
```

```
    //可以看出 av_read_frame读取的是一帧视频，并存入一个AVPacket的结构中
```

```
    if (av_read_frame(pFormatCtx, packet) < 0)
```

```
    {
```

```
        break; //这里认为视频读取完了
```

```
    }
```

```
int64_t realTime = av_gettime() - start_time; //主时钟时间
```

```
while(pts > realTime)
```

```
{
```

```
    msleep(10);
```

```
    realTime = av_gettime() - start_time; //主时钟时间
```

```
}
```

```
//生成图片
```

```
if (packet->stream_index == videoStream)
```

```
{
```

```
    // 解码 packet存在pFrame里面
```

```
    ret = avcodec_decode_video2(pCodecCtx, pFrame, &got_picture, packet);
```

```
    if (ret < 0) {
```

```
        printf("decode error");
```

```
        return ;
```

```
    }
```

然后在解码之后加入语句:

//获取显示时间 pts

```
pts = pFrame->pts = pFrame->best_effort_timestamp;
```

pts *= 1000000 * av_q2d(pFormatCtx->streams[videoStream]->time_base); //绝对时间
加入方法如图

```
{
    msleep(5);
    realTime = av_gettime() - start_time; //主时钟时间
}
// 解码 packet 存在 pFrame 里面
ret = avcodec_decode_video2(pCodecCtx, pFrame, &got_picture, packet);
if (ret < 0) {
    qDebug() << "decode error";
    return ;
}
```

```
//获取显示时间pts
pts = pFrame->pts = pFrame->best_effort_timestamp;

//qDebug() << "dts:" << packet->dts << "pts:" << pts;
//绝对时间
pts *= 1000000 * av_q2d(pFormatCtx->streams[videoStream]->time_base);
```

//有解码器解码之后得到的图像数据都是 YUV420 的格式，而这里需要将其保存成图片文件
//因此需要将得到的 YUV420 数据转换成 RGB 格式

```
if (got_picture) {
    sws_scale(img_convert_ctx,
              (uint8_t const * const *) pFrame->data,
```

其中, pFrame 中 best_effort_timestamp 是基于视频流的时间基的估计的帧时间戳, 可用于同步音频和视频.